

Drydock Launch Readiness and Adoption Plan

Status: Working plan. Check items off in place.

Updated: 2026-08-14

Owner: Ed Barlow

Goal: Launch Drydock with one credible promise, one independently inspectable proof, and a short path from interest to a successful first run.

0. Decision

Drydock does not need more product surface before launch. It needs a smaller public story and a cleaner proof surface.

The weak response in r/SpecDrivenDevelopment is not evidence that Drydock is too small. The venue is small, and the posts asked readers to understand a large methodology before showing one result. The useful signal in the discussion was that readers immediately asked how execution works and whether subscription CLI use is supported. The launch must answer those questions directly and then lead to evidence.

Use this public promise:

Drydock turns a long specification into an ordered agent build and publishes the tests, prompts, changes, and release verdict as a verifiable receipt.

Everything else — SAIL, QuarterDeck, Rigging, Soundings, Sea Trials, enterprise governance, document generation, and the full command surface — supports that promise. It is not the opening pitch.

Launch rule: do not add another example or major feature until one existing example has a boringly consistent verdict, a five-minute guided tour, and a successful clean-room reproduction.

1. Current Position

Completed since the first plan

- ☒ Updated the overview video.
- ☒ Implemented `drydock uat` as a repeatable, end-to-end scored build.
- ☒ Published separate evidence repositories for [CommonMark](#), [Reading List](#), and [TOML](#).
- ☒ Built nontrivial CommonMark and TOML conformance examples.
- ☒ Built the small Reading List example, including an incremental-specification update path.
- ☐ Complete the jq build. This is useful depth, but it is not a launch prerequisite.

Launch blockers visible today

- ☐ Make one proof repo internally consistent. The current public receipts invite avoidable skepticism:
 - CommonMark is headed `PASSED`, but `score ac` and `score release` exit 1.
 - TOML is headed `PASSED`, but `status --check` and `score ac` exit 1.

- Reading List is headed **FAILED** because the refit update exits 1.
- ☐ Explain the distinction between the UAT verdict and advisory command results, or change the verdict contract so a reader never has to infer it. A launch artifact cannot say both “passed” and “release failed” without an explicit, credible explanation.
- ☐ Put proof above documentation in the main README. The current README has no worked-example link, repeats the tagline, presents seven capabilities at once, and still requires several configuration commands before first value.
- ☐ Make each proof kit browsable without cloning. Enable GitHub Pages for the selected repo and set its repository homepage to the rendered **index.html**.
- ☐ Audit every published receipt for credentials, personal paths, private source, provider tokens, licensing, and unexpectedly large raw artifacts.

The public evidence repositories are the strongest work completed since the first plan. They are also now the highest-risk credibility surface. Fix them before sending more traffic.

2. Phase 1 — Establish One Golden Proof

Use **TOML as the initial candidate** because its build and release score pass and its external conformance suite is easy to explain. If CommonMark reaches a fully consistent verdict first, use it instead. Do not present all examples with equal weight.

- ☐ Resolve every contradictory exit in the selected run. The final receipt must show:
 - lifecycle completed;
 - external conformance suite passed;
 - target completion check passed;
 - acceptance score passed;
 - release score passed;
 - integrity verification passed.
- ☐ Rerun from a clean machine or clean VM using the published PyPI release, not the source tree.
- ☐ Record the exact Drydock version, provider, model, platform, elapsed time, LLM calls, token use, repair count, conformance result, and final verdict at the top of the example README.
- ☐ Add a five-minute guided path to the example README:
 1. read the input specification;
 2. inspect the generated Blueprint and Manifest;
 3. inspect the delivered code;
 4. inspect the external test result;
 5. verify **SHA256SUMS**.
- ☐ Add a compact result table before any explanation of UAT internals.
- ☐ Link the exact successful run, not merely the repository root.
- ☐ Publish the generated HTML through GitHub Pages and populate the repository description, homepage, and topics. TOML currently has no description; none of the three repos has a homepage or topics.
- ☐ State what the example does **not** prove: one run is evidence of one run, not a benchmark, general success rate, security certification, or deterministic LLM output.

Exit criterion: a skeptical engineer can verify what was built, how it was tested, and why the verdict is **PASSED** in five minutes without installing Drydock.

Supporting examples

- **CommonMark:** publish second as the scale/conformance example after its release and acceptance results agree with the headline verdict.
 - **Reading List:** keep as the fast onboarding and refit example. Do not promote it while its published latest run is failed. Once fixed, it should become the clean-install quickstart because a four-hour CommonMark run is not an activation path.
 - **jq:** use as a post-launch engineering diary and stress test. Its value is the difficult generator/backtracking model and the honest failure progression, not another green badge.
-

3. Phase 2 — Rebuild the Front Door Around the Proof

The README currently describes the whole product before proving any part of it. Reverse that order.

- ☐ Replace the hero copy with the public promise from Section 0 and one sentence identifying the user: engineers using Claude Code or Codex to deliver projects too large for one prompt.
- ☐ Put three actions immediately below it:
 1. **Inspect a verified build** — the golden proof;
 2. **Try the small example** — Reading List after it passes;
 3. **Understand the method** — documentation or updated video.
- ☐ Add one screenshot of the golden receipt showing the verdict, external test count, elapsed time, model, and integrity check. Use the updated video as secondary depth, not the only proof.
- ☐ Reduce the first screen to three differentiators:
 - dependency-ordered work with bounded context;
 - deterministic acceptance against external tests;
 - inspectable build receipts.
- ☐ Move SAIL and nautical vocabulary below the first proof and installation action.
- ☐ Remove duplicated overview copy and correct visible prose errors before launch, including **Methodology**, **database**, **questionnaires**, and agreement errors.
- ☐ Replace the configuration wall with one copyable quickstart. If configuration cannot be inferred or prompted safely, provide a bootstrap command or a checked example config.
- ☐ Add a prominent beta/status statement and a support route with a response expectation.
- ☐ Add direct links to all public examples in a small “Build receipts” table, with only the golden proof labelled recommended.
- ☐ Keep the comparison matrix available, but do not make it the primary call to action.

Exit criterion: after ten seconds, a reader knows the problem Drydock solves and can open a real successful receipt with one click.

4. Phase 3 — Release Hygiene

Do this once, immediately before launch. Freeze non-blocking feature work while it runs.

- ☐ Cut a release candidate and run the complete test, lint, format, package, and installed-wheel verification contracts.
- ☐ Test clean installation and the small example on Linux, macOS, and Windows/WSL, or publish an explicit tested-platform matrix with unsupported combinations named.

- ☐ Test both supported subscription providers. Publish observed compatibility; do not infer terms-of-service conclusions or promise that subscription policies will remain unchanged.
- ☐ Verify every README, PyPI, project-site, video, comparison-matrix, issue, and example link.
- ☐ Confirm package and project naming is unambiguous. Another active package uses the `drydock-cli` name; use `drydock-sdd` consistently in search-facing copy and metadata.
- ☐ Verify that the PyPI description shows the same promise, quickstart, proof link, supported Python versions, license, repository, issue tracker, and documentation links as GitHub.
- ☐ Review open issues and known failures. Publish a short `KNOWN_LIMITATIONS.md` or equivalent; hiding normal beta limitations costs more trust than naming them.
- ☐ Tag the release, publish it, install that exact artifact in a blank environment, and run the Reading List smoke path again.
- ☐ Freeze the successful proof receipt. Never silently replace it; publish later runs alongside it with their Drydock versions.

Exit criterion: the artifact a stranger installs is the artifact used by the quickstart and the proof can be reproduced from published inputs.

5. Phase 4 — Five Design-Partner Runs

Do this before a broad Show HN launch. The next required evidence is not another self-run; it is a stranger completing a run.

- ☐ Recruit five senior engineers who already use Claude Code or Codex and have a specification, PRD, migration plan, or substantial issue set.
- ☐ Ask each person to bring their own project. Do not lead with the Drydock terminology.
- ☐ Observe the first install and import silently. Record:
 - time to first useful output;
 - first confusing screen or term;
 - first command failure;
 - whether they reached a completed build;
 - whether they returned for a second run;
 - whether they trusted the receipt.
- ☐ Fix any repeated activation failure before recruiting the next cohort.
- ☐ Ask for permission to publish one anonymized case study, including failure and repair history.
- ☐ Obtain three externally authored issue reports or discussion threads before broad launch.

Recruit directly from former colleagues, senior engineering contacts, and people who engaged substantively on Reddit. A personal request for a 45-minute observed trial is more likely to produce evidence than another general announcement.

Exit criterion: three of five partners finish the small path, one tries a real project, and the same onboarding defect does not stop two consecutive participants.

6. Phase 5 — Publish Useful Artifacts, Then Launch

The next posts should be technical artifacts with Drydock as the implementation, not repeated announcements.

Publication order

- ☐ **Anatomy of a verifiable agent build receipt.** Walk through the golden proof from source specification to external test and checksum. This is the new canonical article.
- ☐ **The failed Reading List refit.** Explain what failed, how the receipt exposed it, and what changed. A concrete failure analysis demonstrates the evidence claim better than a flawless demo.
- ☐ **Building a CommonMark parser from the specification.** Lead with the conformance result and exact cost/time, then show where planning and repair mattered.
- ☐ **Building jq: where the one-value execution model broke.** Publish the progression even if jq does not reach full conformance.
- ☐ **Comparison matrix update.** Keep it factual, date-stamped, linked to primary sources, and explicit about where alternatives win.
- ☐ **Show HN.** Update `ideas/SHOW_HN_POST.md` around `drydock uat` and the golden proof before submission. Remove claims that no longer match the implementation or provider policies.

Channel order

Channel	Use	Gate
Direct outreach	Observed design-partner runs	Golden proof live
LinkedIn	Receipt walkthrough and engineering case study	One external run
r/SpecDrivenDevelopment	Failure analysis or conformance result	Artifact published
r/ClaudeAI / relevant Codex community	Provider-specific workflow and result	Tested provider matrix
Hacker News	Show HN linking the repository and golden proof	Three completed external trials
Lobste.rs / r/programming	Deep technical post only	Measured result and reproducible repo

Do not cross-post identical copy. Do not lead with “specification-driven development” outside its own community; lead with the observable failure: long agent builds lose decisions, redo work, and cannot prove completion.

7. Six-Week Operating Cadence

Ship one product correction and one public artifact per week. Do not make six announcements.

Week	Product evidence	Public artifact
1	Coherent golden proof and GitHub Pages	Proof-receipt walkthrough
2	Reading List clean install and passing refit	Failed-refit postmortem
3	First two observed partner runs	CommonMark case study

Week	Product evidence	Public artifact
4	Onboarding corrections and third partner run	Updated comparison matrix
5	Five partner runs complete	jq engineering diary
6	Release candidate reproduced from PyPI	Show HN

Delay the public launch if the proof or clean install fails. Do not delay it merely to finish jq, add another command, polish every document, or obtain a larger benchmark suite.

8. Measurement

Track the funnel weekly in one append-only table. Raw stars and impressions are context, not the goal.

Stage	Metric	Source	Initial target
Reach	Qualified visits to proof and quickstart	GitHub Insights / site analytics	Establish baseline
Interest	Proof-kit opens or Pages visits	Pages analytics or server logs	20
Activation	Clean installs started	Direct trials / issue template	10
First value	Successful Reading List runs by others	Submitted receipt or issue	5
Real use	User-owned projects imported	Interview / discussion	3
Retention	A second run or refit by the same user	Interview / submitted receipt	2
Advocacy	External issue, write-up, or referral	GitHub / web	1

- ☐ Create the weekly measurement note before the first artifact is published.
- ☐ Add optional "share this receipt" instructions instead of anonymous telemetry for the first cohort.
- ☐ Decide on telemetry only after observed trials show which event would change a decision.

Launch success is not a front page or star count. It is one engineer using Drydock on their own project, returning for a second run, and being able to explain why they trusted or rejected the receipt.

9. Not Doing Before Launch

- More UAT targets after jq.
- A Discord server.
- Paid advertising.
- A broad benchmark claiming superiority from one run per tool.
- Renaming the nautical vocabulary.
- Rewriting the product around Reddit feedback from people who did not run it.
- Adding providers without a design partner who needs one.
- Posting the same announcement repeatedly in a small subreddit.
- Claiming enterprise readiness before external users have completed the workflow.

10. Exact Next Order

1. Finish the current jq run only far enough to capture its result and lessons; do not make full jq conformance a launch gate.
2. Select TOML or CommonMark as the golden proof and eliminate every verdict contradiction.
3. Publish that proof through GitHub Pages with a result-first README and exact-run link.
4. Rewrite the main README first screen around the proof and repair the clean-install path.
5. Fix and republish Reading List until its initial build and refit both pass from the PyPI package.
6. Run the release-hygiene checklist and cut the release candidate.
7. Conduct five observed design-partner trials.
8. Publish the proof walkthrough, then the failure postmortem, then launch broadly.

The strategic shift is from **“explain the methodology to a large audience”** to **“let a small number of qualified engineers inspect one result and reproduce it.”** Drydock is already large enough. The launch surface must become smaller than the product.